

Week 2 Module:

Date: 17/06/2026

```
//1. Fraud Transaction Detection (Linear Search)

//ALGORITHM
// Start
// Read the number of transactions N.
// Store the N transaction IDs in an array.
// Read the suspicious transaction ID (key).
// Set found = 0.
// Repeat for each element in the array from index 0 to N-1:
// If array[i] == key:
// Set found = 1.
// Exit the loop.
// If found == 1:
// Print "Fraud Transaction Found".
// Otherwise:
// Print "Transaction Not Found".
// Stop.
```

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;

    int transactions[N];

    // Input transaction IDs
    for (int i = 0; i < N; i++) {
        cin >> transactions[i];
    }

    int suspiciousID;
    cin >> suspiciousID;

    bool found = false;

    // Linear Search
    for (int i = 0; i < N; i++) {
        if (transactions[i] == suspiciousID) {
            found = true;
            break;
        }
    }

    if (found)
        cout << "Fraud Transaction Found";
    else
        cout << "Transaction Not Found";

    return 0;
}
```

```
//2.Algorithm: Server Log Search using Binary Search
```

```
//Start
//Read the number of log IDs N.
//Store the N sorted log IDs in an array.
//Read the target log ID (key).
//Initialize:
//low = 0
//high = N - 1
//found = false
//Repeat while low <= high:
//Calculate mid = (low + high) / 2.
//If array[mid] == key:
//Set found = true.
//Exit the loop.
//Else if key < array[mid]:
//Set high = mid - 1.
//Else:
//Set low = mid + 1.
//If found == true, print "Log Found".
//Otherwise, print "Log Not Found".
//Stop.
```

```
int main() {
    int N;
    cin >> N;

    int logs[N];

    // Input sorted log IDs
    for (int i = 0; i < N; i++) {
        cin >> logs[i];
    }

    int target;
    cin >> target;

    int low = 0, high = N - 1;
    bool found = false;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (logs[mid] == target) {
            found = true;
            break;
        }
        else if (target < logs[mid]) {
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }

    if (found)
        cout << "Log Found";
    else
        cout << "Log Not Found";
}
```

```
//3. Algorithm: Sort Employee Salaries using Bubble Sort

//Start
//Read the number of employees N.
//Store the salaries in an array.
//Repeat N - 1 times:
//Compare each pair of adjacent elements.
//If the current salary is greater than the next salary, swap them.
//Print the sorted salaries.
//Stop.
```

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;

    int salary[N];

    // Input salaries
    for (int i = 0; i < N; i++) {
        cin >> salary[i];
    }

    // Bubble Sort
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - 1 - i; j++) {
            if (salary[j] > salary[j + 1]) {
                int temp = salary[j];
                salary[j] = salary[j + 1];
                salary[j + 1] = temp;
            }
        }
    }

    // Print sorted salaries
    for (int i = 0; i < N; i++) {
        cout << salary[i] << " ";
    }

    return 0;
}
```

```
// 4.Algorithm: Sort Website Response Time using Insertion Sort

//Start
//Read the number of response times N.
//Store the response times in an array.
//Repeat from the second element (i = 1) to the last element:
//Store the current element as key.
//Compare key with the elements before it.
//Shift all larger elements one position to the right.
//Insert key into its correct position.
//Print the sorted response times.
//Stop.
```

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;

    int response[N];

    // Input response times
    for (int i = 0; i < N; i++) {
        cin >> response[i];
    }

    // Insertion Sort
    for (int i = 1; i < N; i++) {
        int key = response[i];
        int j = i - 1;

        while (j >= 0 && response[j] > key) {
            response[j + 1] = response[j];
            j--;
        }

        response[j + 1] = key;
    }

    // Print sorted response times
    for (int i = 0; i < N; i++) {
        cout << response[i] << " ";
    }

    return 0;
}
```

```
// 5. Algorithm: Prioritize Bug Reports using Selection Sort
//Start
//Read the number of bug reports N.
//Store the priority scores in an array.
//Repeat from the first element to the second-last element:
//Assume the current element is the minimum.
//Compare it with the remaining elements.
//Find the index of the smallest element.
//Swap the current element with the smallest element.
//Print the sorted priority scores.
//Stop.
```

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;

    int priority[N];

    // Input priority scores
    for (int i = 0; i < N; i++) {
        cin >> priority[i];
    }

    // Selection Sort
    for (int i = 0; i < N - 1; i++) {
        int min = i;

        for (int j = i + 1; j < N; j++) {
            if (priority[j] < priority[min]) {
                min = j;
            }
        }

        int temp = priority[i];
        priority[i] = priority[min];
        priority[min] = temp;
    }

    // Print sorted priority scores
    for (int i = 0; i < N; i++) {
        cout << priority[i] << " ";
    }

    return 0;
}
```